

Alon Anthony ★★★★★
3453 Wheeling Drive, 95051, Santa Clara CA , United States,
webmaster@synpat.com,
(408) 480 31 86,
www.ip-consult.net,

Report of Evidence of Use

The views presented here are of the author only and do not necessarily reflect the views of others. The author is an independent contractor that acts on behalf of neither the patent owner nor any other third party, including ClaimCharts.info. We would like to thank ClaimCharts.info for introducing to us the opportunity to prepare this report. For comments and questions, please feel free to contact us directly, or via my profile at ClaimCharts.info.

Patent Number: USUS8909824

Claim Number: (1

1. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter, the receiver comprising: 1. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter, the receiver comprising: a first module for processing the fragments according to a processing index in each of the fragments, wherein the each of the fragments has one or more data blocks, the processing index in the each of the fragments is to facilitate the fragments to be processed sequentially, the each of the fragments is processed till one or more of predefined conditions is met, the predefined conditions include: the timer assigned to the each of the fragments is timed out or an acknowledgement of at least one of the data blocks in the each of the fragments is received; and a second module designed to start the timer assigned to the each of the fragments, wherein the timer has a timeout value depending on a life of the each of the fragments and time parameters thereof , wherein the timer is running until processing of the each of the fragments is terminated, the timer is then reassigned to a next one of the fragments before the next one of the fragments is processed.

Element 1:

1. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter, the receiver comprising:

CLOCKLESS VIRTUAL GPIO

Application number: 20150149672
Abstract: A virtual GPIO architecture for an integrated circuit is provided that both serializes virtual GPIO signals and deserializes virtual GPIO signals without the need for an external clock.
Type: Application
Filed: November 13, 2014
Issued: May 28, 2015
Inventors: Lalan Jee Mishra, Richard Dominic Wietfeldt, Mohit Kishore Prasad

SYSTEMS AND METHODS FOR BIASING A BUS

Application number: 20150149671
Abstract: A bi-directional differential bus interface that includes a differential transmitter having a non-inverting terminal and an inverting terminal, a differential receiver having a non-inverting terminal and an inverting terminal, and a biasing circuit that is electrically coupled to the non-inverting terminal of the differential transmitter and the inverting terminal of the differential transmitter. The biasing circuit is configured to generate a voltage between the non-inverting terminal of the differential transmitter and the inverting terminal of the differential transmitter that is approximately 200 mV or more in response to assertion of a control signal received on a control input of the biasing circuit.
Type: Application
Filed: May 21, 2012
Issued: May 28, 2015
Assignee: SCHNEIDER ELECTRIC IT CORPORATION
Inventors: Daniel J. Rohr, Richard Earl Jenkins

METHOD AND APPARATUS FOR USE IN A DATA PROCESSING SYSTEM

Application number: 20150143005
Abstract: A method is described, for use in a data processing system, the system having a node and a communication link, wherein the communication link is coupled to the node. The method can comprise obtaining first digital signal information associated with a first signal, transmitting the first signal from the node to the communication link, receiving a second signal from the communication link at the node, and analysing the second signal to obtain second digital signal information. The method can further include combining first digital signal information with second digital signal information and flagging a combination outside a predetermined condition space. Further, an apparatus, for use in the data processing system is described that can be operative to perform the method. A data processing system is also described.
Type: Application
Filed: November 18, 2013
Issued: May 21, 2015
Assignee: Infineon Technologies AG
Inventor: Achim VOWE

MULTI-PROTOCOL COMBINED RECEIVER

Application number: 20150143006
Abstract: A receiver circuit configured to operate in a DisplayPort (DP) mode and a High-Definition Multimedia Interface (HDMI) mode. The receiver circuit includes: termination circuitry configured to receive a DP signal in the DP mode and an HDMI signal in the HDMI mode; and voltage common-mode (VCM) level shifter circuitry configured to operate as a pass-through for the DP signal in the DP mode and generate a converted HDMI signal from the HDMI signal in the HDMI mode.
Type: Application
Filed: November 19, 2013
Issued: May 21, 2015
Assignee: Synaptics Incorporated
Inventor: Yonggang Chen

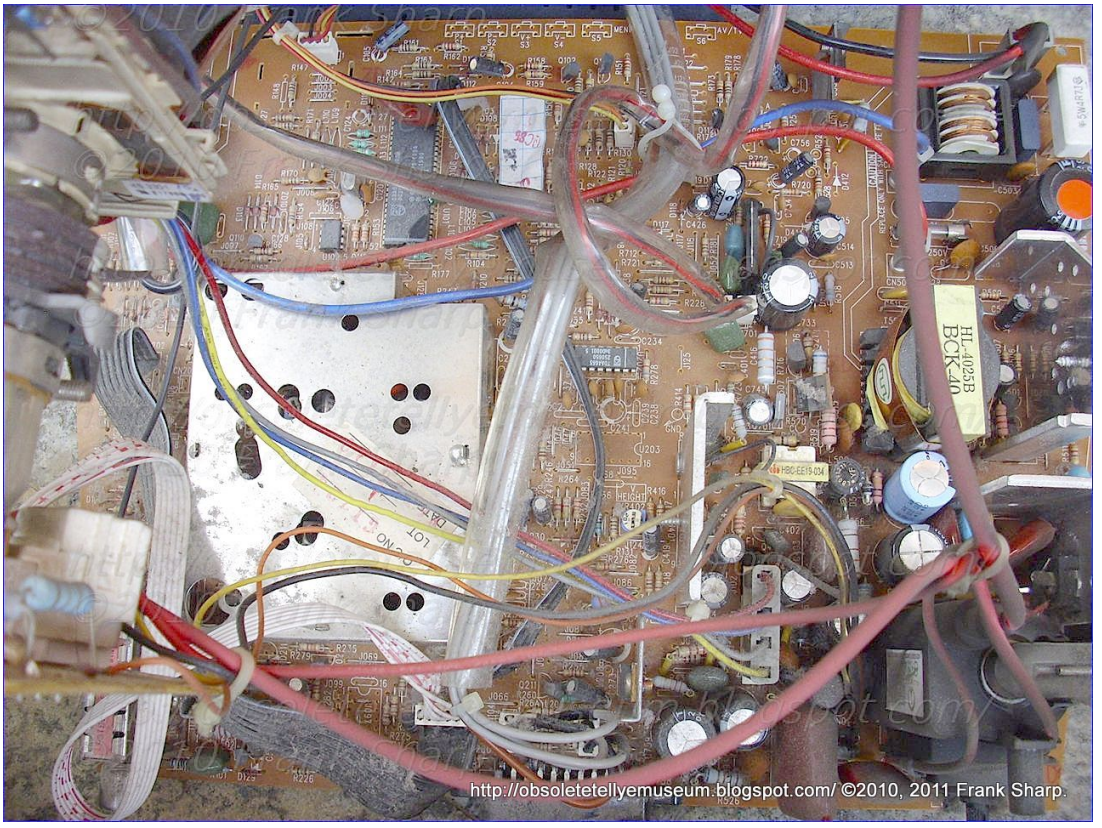
METHOD FOR SYNCHRONIZING DISPLAY ELEMENTS IN A CONTROL SYSTEM FOR AUTOMATED CONTROL OF A TECHNICAL INSTALLATION

Application number: 20150134865
Abstract: A method for synchronizing display elements of a plurality of subscribers of a distributed, bus-based control system is proposed. The subscribers comprise a message transmitter and a plurality of message receivers. The message receivers each comprise a respective display element, which can be toggled between a switched-on state and a switched-off state, and a control unit for

producing a control signal for toggling the display element from one state to the other. The message transmitter sends a respective current bus message via the bus in defined recurring bus message time intervals. The respective bus message comprises a bus message counter that has a respective value from a defined value range. The counter value is changed in a predefined manner each time when a current bus message is sent. The message receivers receive the respective bus messages via the bus.
Type: Application
Filed: January 19, 2015
Issued: May 14, 2015
Inventors: Thorsten GODAU, Christian ADAMS, Reinhard SPERRER, Dietmar SEIZINGER

SUPERSPEED INTER-CHIP INTERFACE

Application number: 20150134866
Abstract: An interface for low power, high bandwidth communications between units in a device in provided herein. The interface comprises a USB 3.0 system interface and a SuperSpeed inter-chip (SSIC) protocol adaptor configured to facilitate communications between the USB 3.0 system interface and an M-PHY interface.
Type: Application
Filed: January 20, 2015
Issued: May 14, 2015
Inventors: Sridharan Ranganathan, David J. Harriman, Anoop Mukker, Satheesh Chellappan, Karthi R. Vadivelu, Shalini Sharma, Zeeshan Sarwar



Element 2:

1. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter, the receiver comprising:

To created a dynamic and multi-pane user interface on Android, you need to encapsulate UI components and activity behaviors into modules that you can swap into and out of your activities. You can create these modules with the `Fragment` class, which behaves somewhat like a nested activity that can define its own layout and manage its own lifecycle.

When a fragment specifies its own layout, it can be configured in different combinations with other fragments inside an activity to modify your layout configuration for different screen sizes (a small screen might show one fragment at a time, but a large screen can show two or more).



Element 3:

a first module for processing the fragments according to a processing index in each of the fragments,

```
<http://data.linkedtv.eu/annotation/Anno_199_FhG_ASR>
  a oa:Annotation , opmv:Artifact ;
  opmv:wasGeneratedAt "2012-06-29T18:19:35.153Z"^^xsd:dateTime ;
  opmv:wasGeneratedBy
    [ a opmv:Process ;
      opmv:wasPerformedBy <http://data.linkedtv.eu/organization/FhG>
    ];
  linkedtv:confidence "0.234"^^xsd:float ;
  oa:hasTarget <http://data.linkedtv.eu/media/TUSSEN_KUNST_AVR000080E2#t=1195,2312> ;
  oa:hasBody <http://data.linkedtv.eu/asr/TUSSEN_KUNST_AVR000080E2_asr_01> .
```

– The instance of the class `linkedtv:ASR` that is being referred in the previous annotation enables to store the string containing the transcription.

```
<http://data.linkedtv.eu/asr/TUSSEN_KUNST_AVR000080E2_asr_01> a linkedtv:ASR ;
  rdfs:label "Wij zijn zo gewend dat dingen gedrukt zijn Dat iedereen zegt: O, Indiase prenten! Maar
      het zijn geen prenten. Het zijn Indiase schilderingen. U bent in India geweest?Ja. Maar ik
     heb deze daar niet gekocht. Deze kocht ik op de veiling in Amsterdam. 15 jaar geleden. Nou, ik
      weet niet wat u betaald heeft, dat wil ik ook niet weten... Niet veel.Maar dat heeft u goed
     gedaan, denk ik. Want deze schildering... Een heel mooi vorstelijk portret. Het komt uit Noord
     -India. Hier heeft u een Indiaas miniatuur. Dat is ook uit het noorden. Ze zit daar prachtig
     op een mooie stoel. U ziet al dat het veel flamboyanter is dan de andere. Die doeken en dingen
      gaan al veel meer opzij. Dat vind je ook in het gebied van Jodhpur, Udaipur. Aan je goud zie
     je dat het een tamelijk late miniatuur is. Ze leggen er ook kleine pareltjes op. Die geven
     relief. En dat zit je al in de 19e eeuw. Als je goed kijkt heft het hier een kleine
     beschadiging. Miniaturen horen eigenlijk puntgaaf te zijn. Je moet denken aan 650 euro. Wel
     een hele mooie vondst. Welke periode is dit ongeveer? Ongeveer 1740. Hij is iets beschadigd.
     Dat heeft met de waarde te maken. Deze miniatuur: 1250 euro." .
```


Building a Dynamic UI with Fragments

To create a dynamic and multi-pane user interface on Android, you need to encapsulate UI components and activity behaviors into modules that you can swap into and out of your activities. You can create these modules with the `Fragment` class, which behaves somewhat like a nested activity that can define its own layout and manage its own lifecycle.

When a fragment specifies its own layout, it can be

Get started

Dependencies and prerequisites

- Basic knowledge of the Activity lifecycle (see Managing the Activity Lifecycle)
- Experience building XML layouts

configured in different combinations with other fragments inside an activity to modify your layout configuration for different screen sizes (a small screen might show one fragment at a time, but a large screen can show two or more).

This class shows you how to create a dynamic user experience with fragments and optimize your app's user experience for devices with different screen sizes, all while continuing to support devices running versions as old as Android 1.6.

You should also read

- Fragments
- Supporting Tablets and Handsets

Try it out

DOWNLOAD THE SAMPLE

FragmentBasics.zip

Lessons

Creating a Fragment
Learn how to build a fragment and implement basic behaviors within its callback methods.

Building a Flexible UI
Learn how to build your app with layouts that provide different fragment configurations for different screens.

Element 4:

wherein the each of the fragments has one or more data blocks,

Master bedroom for $60/$55 a night, ($65 for double occupancy) after August 21st. The small bedroom $55/$50 begins June 24th to June 27th and then after August 21st. The dates of July 26th through Aug*ust 21st. are no longer av*ailable. I am looking for long term renters after August 21st for the entire school year.

Nestled in the North Berkeley foothills, 5 blocks from UC Berkeley, and just above the popular culinary haven, the \"Gourmet Ghetto\" is a charming bed and breakfast which continues to delight guests with tasteful details such as hand blown glass lamps, antique furnishings, and petrified wood sink & counters. This vine covered home, combining old world elegance with state of the art convenience, is located on a quiet street, and although it is small in size, it has two stories, with rooms that are light-filled and cozy .

The larger bedroom has a partial bay view and is furnished with an organic double bed mattress, a cedar armoire, a 200 year old bench and a desk and chair. The second bedroom, which also has a partial bay view, is furnished with an organic single futon on the floor and a small table. Located next to the bedrooms are a travertine marble bathroom and laundry facility.

The price includes an organic continental breakfast and kitchen privileges for the rest of your meals. The kitchen is a beautifully designed work of art with maple cupboards, a Bosch dishwasher, and Miele cooktop.

The floors are hardwood throughout, and there is radiant floor heat for your comfort.

Guests may also enjoy a back yard patio with out door eating during the summer months.

I appreciate guests who share my interest in environmentally friendly products. No smoking. I do have one cat so no one with cat allergies. I will be sleeping on the upstairs couch.

A recent front facing photo of you will be greatly appreciated.

do NOT contact me with unsolicited services or offers

Element 5:

the processing index in the each of the fragments is to facilitate the fragments to be processed sequentially,

When a file system is first initialized on a partition (the partition is formatted for the file system), the partition contains only a few small internal structures and is otherwise one contiguous block of empty space.[1] This means that the allocator algorithm is completely free to place newly created files anywhere on the partition. For some time after creation, files on the file system can be laid out near-optimally. When the operating system and applications are installed or other archives are unpacked, laying out separate files sequentially also means that related files are likely to be positioned close to each other.

However, as existing files are deleted or truncated, new regions of free space are created. When existing files are appended to, it is often impossible to resume the write exactly where the file used to end, as another file may already be allocated there — thus, a new fragment has to be allocated. As time goes on, and the same factors are continuously present, free space as well as frequently appended files tend to fragment more. Shorter regions of free space also mean that the allocator is no longer able to allocate new files contiguously, and has to break them into fragments. This is especially true when the file system is more full — longer contiguous regions of free space are less likely to occur.

Note that the following is a simplification of an otherwise complicated subject. The method which is about to be explained has been the general practice for allocating files on disk and other random-access storage for over 30 years. Some operating systems do not simply allocate files one after the other, and some use various methods to try to prevent fragmentation, but in general, sooner or later, for the reasons explained in the following explanation, fragmentation will occur as time goes by on any system where files are routinely deleted or expanded. Consider the following scenario, as shown by the image on the right:



Some early file systems were unable to fragment files. One such example was the Acorn DFS file system used on the BBC Micro. Due to its inability to fragment files, the error message *can't extend* would at times appear, and the user would often be unable to save a file even if the disk had adequate space for it.

DFS used a very simple disk structure and files on disk were located only by their length and starting sector. This meant that all files had to exist as a continuous block of sectors and fragmentation was not possible. Using the example in the table above, the attempt to expand file F in step five would have failed on such a system with the *can't extend* error message. Regardless of how much free space might remain on the disk in total, it was not available to extend the data file.

Standards of error handling at the time were primitive and in any case programs squeezed into the limited memory of the BBC Micro could rarely afford to waste space attempting to handle errors gracefully. Instead, the user would find themselves dumped back at the command prompt with the *Can't extend* message and all the data which had yet to be appended to the file would be lost. The resulting frustration would be greater if the user had taken the trouble to check the free space on the disk beforehand and found free space. While free space on the disk may exist, the fact that it was not in the place where it was needed was not apparent without mentally analysing the numbers presented by the disk catalog and so would escape the user's notice. In addition, DFS users had almost without exception previously been accustomed to cassette file storage, which does not suffer from this error. The upgrade to a floppy disk system was expensive, but it freed users from the unreliability and terrible slowness of the cassette system, and it was a shock to make the sudden and unpleasant discovery that the upgrade might without warning cause data loss of their recent work.[3][4]

Element 6:

the each of the fragments is processed till one or more of predefined conditions is met,



We claim:
1. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter, the receiver comprising: a first module for processing the fragments according to a processing index in each of the fragments, wherein the each of the fragments has one or more data blocks, the processing index in the each of the fragments is to facilitate the fragments to be processed sequentially, the each of the fragments is processed till one or more of predefined conditions is met, the predefined conditions

include: the timer assigned to the each of the fragments is timed out or an acknowledgement of at least one of the data blocks in the each of the fragments is received; and a second module designed to start the timer assigned to the each of the fragments, wherein the timer has a timeout value depending on a life of the each of the fragments and time parameters thereof, wherein the timer is running until processing of the each of the fragments is terminated, the timer is then reassigned to a next one of the fragments before the next one of the fragments is processed.

2. The receiver of claim 1, wherein each of the fragments has a fragment life and a processing time, the timer assigned to the each of the fragments has an assignment time, and the timeout value is a function of the fragment life, the processing time, and the assignment time.

3. The receiver of claim 1, wherein the predefined conditions further include that a negative acknowledgement for the each of the fragments is received.

4. The receiver of claim 1, wherein the predefined conditions further include that the each of the fragments is deleted.

5. The receiver of claim 1, wherein the predefined conditions further include that a duplicate one of the each of the fragments happens.

6. The receiver of claim 1, wherein the each of the fragments has a start Block Sequence Number (BSN) and there is a window start BSN, and the pre-defined conditions further include that the start-BSN of the each of the fragments is less than or equal to the window-start BSN.

7. The receiver of claim 1, wherein both the receiver and the transmitter are engaged in wireless communication.

8. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter in transporting the data fragments, the receiver comprising: a first module for processing the fragments according to a processing index in each of the fragments, wherein the each of the fragments has one or more data blocks, the processing index in the each of the fragments is to facilitate the fragments to be processed sequentially; a second module for calculating a timeout value of the timer and assigning the timer having a timeout value to the each of the fragments, wherein the timer has a timeout value depending on a life of the each of the fragments and time parameters thereof; a storage device; a third module for inserting identifying information of the each of the fragments in a first storage location of the storage device and pointing a header pointer to the first storage location, and inserting the identifying information of a next fragment in a next storage location of the storage device, and the third module assigning the timer to the next fragment and moving the header pointer to the next storage location for the next fragment when one or more of pre-defined conditions is met.

9. The receiver of claim 8, wherein the predefined conditions include: the timer assigned to the each of the fragments is timed out or an acknowledgement of at least one of the data blocks in the each of the fragments is received.

10. The receiver of claim 8, wherein the timer is used as a retry timer when the fragments are being transmitted from the transmitter to the receiver, or the timer is used as a purge timer when the fragments are being received in the receiver from the transmitter.

11. The receiver of claim 8 further including a timer table structured to store identifying information of each of the fragments, the timer table provided to facilitate the timer to be used for each of the fragments in the connection being specified between the transmitter and the receiver.

12. The receiver of claim 11, wherein the identifying information of each of the fragments includes an identifying address, a start address and an address in the timer table.

13. The receiver of claim 12, wherein the identifying information of each of the fragments further includes a start block sequence number (BSN) thereof, a processing index thereof, and a pointer to lead to further information of the each of the fragments.

14. The receiver of claim 8, wherein the pre-defined conditions further include that: the timer assigned to the each of the fragments is timed out in transmitting the each of the fragments from the transmitter to the receiver; an acknowledgement of receipt for the each of the fragments is received; a negative acknowledgement for the each of the fragments is received; the each of the fragments is deleted; or a duplicate fragment for the each of the fragments is received.

15. A receiver for processing data fragments from a transmitter using a single timer for one connection between the receiver and the transmitter in transporting the data fragments, the receiver comprising: a first module for receiving a fragment, wherein the fragment includes one or more data blocks and has identifying information; a second module for calculating a timeout value of the timer and assigning the timer to the fragment, wherein the timer depends on a life of the fragment and time parameters thereof; a third module for inserting the identifying information in a node of a timer management chain, and inserting the identifying information of a next fragment in a next node of the timer management chain when the next

fragment is received, wherein the third module is designed to remove the node having the identifying information for the fragment with the assigned timer from the timer management chain, and assigning the timer to the next fragment in the next node when one or more pre-defined conditions is met.
16. The receiver of claim 15, wherein the predefined conditions include: the timer assigned to the each of the fragments is timed out or an acknowledgement of at least one of the data blocks in the each of the fragments is received.
17. The receiver of claim 15, wherein the timer is used as a retry timer when the fragments are being transmitted from the transmitter to the receiver, or the timer is used as a purge timer when the fragments are being received in the receiver from the transmitter.
18. The receiver of claim 15, further including a timer table structured to store identifying information of each of the fragments, the timer table provided to facilitate the timer to be used for each of the fragments in the connection being specified between the transmitter and receiver.
19. The receiver of claim 18, wherein the identifying information of each of the fragments includes an identifying address, a start address and an address in the timer table.
20. The receiver of claim 19, wherein the pre-defined conditions include: the timer assigned to the fragment is timed out in transmitting the fragment from a transmitter to a receiver; an acknowledgement of receipt for the fragment is received; a negative acknowledgement for the fragment is received; the fragment is deleted; or a duplicated version of the fragment is received.

Element 7:

the predefined conditions include: the timer assigned to the each of the fragments is timed out

[0002] In wireless transmission using a standard-based technology such as WiMAX, Automatic Repeat-reQuest (ARQ) is an error control method that utilizes acknowledgements and timeouts to achieve reliable data transmission and reception. Each connection is identified by a connection identification (CID) and allows the transmission of multiple service data units (SDUs). Each SDU may be fragmented or divided into one or more data blocks ("BLKs"), where each BLK is identified by a Block Sequence Number (BSN). During transmission, one or more data blocks are grouped into fragments ("FRs") that are first transmitted in sequence in the order of their BSN to the receiver to be reassembled. Each fragment can have one or more sequentially ordered data blocks that are identified by their BSN.
[0003] An index may be used to identify a particular CID, SDU, FR, or BLK herein. Thus, for example, a CIDi may represent the ith connection, an SDUi may represent the ith service data unit, an FRi may represent the ith fragment, and a BLKi may represent the ith data block.
[0004] An FR can be transmitted more than once if the transmitter does not receive an acknowledgement before the retry timer for the block times out. The first time a fragment is transmitted is referred to herein as the first transmission for the fragment.
[0005] FIG. 1 is a representation of an example of an organization of the data for transmission and the transmission sequence for the fragments as defined by IEEE 802.16 for WiMAX. The data to be transmitted has "i" connections represented by the CID list, CID1 through CIDi. The first connection with CID1 has "k" SDUs represented by an SDU list, SDU1 through SDUk. An SDU1 is fragmented into "m" fragments represented by FR1 through FRm while SDU2 is fragmented into "n" fragments represented by FR1 through FRn.
[0006] Each FR within an SDU is first transmitted sequentially. The FR in different SDUs is also first transmitted in the arriving order of the SDU. The dotted arrows in FIG. 1 illustrate the order that FRs are first transmitted. All FRs in SDU1 are first transmitted sequentially, and then the data blocks in SDU2 are first transmitted sequentially. This pattern continues for all the SDUs in a connection.
[0007] An acknowledgment is a message sent by the receiver to the transmitter to indicate that it has correctly received an FR or some FRs.
[0008] Typically, each FR to be transmitted has two timers, a life timer that starts when the FR is first transmitted and a retry timer that starts every time the FR is transmitted. The life timeout value for the life timer is greater than the retry timeout value for the retry timer. The retry timer controls the retransmission of an FR if the transmitter has not received the acknowledgement of that FR before the timeout of the retry timer. After an FR is transmitted, the transmitter waits for an acknowledgement of receipt of the FR from the receiver. If an acknowledgment is not received by the transmitter when the retry timer times out, the retry

timer is restarted and the transmitter re-transmits the FR. This process is repeated until the life timer for the FR times out. When a life-timer of an FR times out, the transmitter sends a discard message to the receiver for that particular FR. On receiving the discard message, the receiver may discard the FR and all preceding FRs, i.e., FRs preceding in transmission sequence to the particular FR, and send an acknowledgement message for the FR and all the proceeding FRs. After receiving the acknowledgement, the transmitter may then delete that FR and the entire previous FRs. Since FRs in an SDU are first transmitted in time order, i.e., sequentially, and by the arriving order of the SDUs, and all FRs have the same total lifetime value, the life timer for the FRs will time out at the same order that they are first transmitted. See IEEE Std 802.16-2004, IEEE Standard for Local and Metropolitan Area Networks-Part 16: Interface for Fixed Broadband Wireless Access Systems, October 2004, and IEEE P802.16e-2005, Draft IEEE Standard for Local and metropolitan area networks Part 16: Air Interface for Fixed and Mobile Broadband Wireless Access Systems, Amendment for Physical and Medium Access Control Layers for Combined Fixed and Mobile Operation in Licensed Bands, and Corrigendum 1, February 2006.
[0009] The number of timers in a system is a provisioned number in an operating system. Each timer has a timer identification and a corresponding timer data structure. To open or close a timer, the operating system must undergo an inserting or deleting procedure from a timer link list that is operating system specific. Therefore, a system that requires the use of a large number of timers is expensive, both in terms of data memory and the operation of the timers.
[0010] Current methods for managing data blocks of fragments require the use of two timers for each data block or fragments. As defined by IEEE 802.16, the maximum number of fragments in the transmission window for a CID connection is 1024. Therefore, the maximum number of timers needed for transmission is 2*1024 for each CID connection. The number of timers required for even few CID connections is therefore very large. Using existing methods, the operating system has to be prepared to reserve a significant amount of its resource for the timers and their operations.
[0011] During reception, a fragment FRi, can be characterized by the following: the index "i", its arrival sequence number that is an indication of its sequence of arrival; the start BSNi or BSNi of FRi, the lowest BSN of the BLKs of Fri; and, ti, the time of its arrival. Therefore, FR1 is the first fragment to arrive while FR2 is the second fragment to arrive. For the two fragments FRi and FRj, if j>i, then tj>ti. Although fragments are transmitted sequentially in the order of their start BSN, they do not necessarily arrive in order due to loss and retransmission. Thus, BSNi may or may not be lower than BSNj.

Read more: http://www.faqs.org/patents/app/20150092725#ixzz3elBr95do

Element 8:

or an acknowledgement of at least one of the data blocks in the each of the fragments is received;

```
This document obsoletes RFC 2960 and RFC 3309.  It describes the
  Stream Control Transmission Protocol (SCTP).  SCTP is designed to
  transport Public Switched Telephone Network (PSTN) signaling messages
  over IP networks, but is capable of broader applications.

  SCTP is a reliable transport protocol operating on top of a
  connectionless packet network such as IP.  It offers the following
  services to its users:

  --  acknowledged error-free non-duplicated transfer of user data,

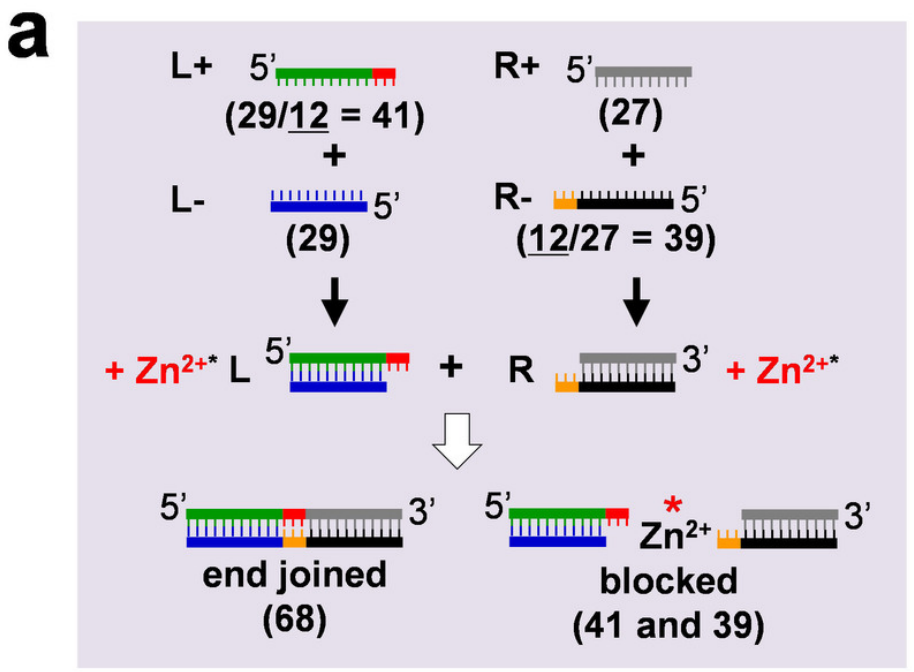
  --  data fragmentation to conform to discovered path MTU size,

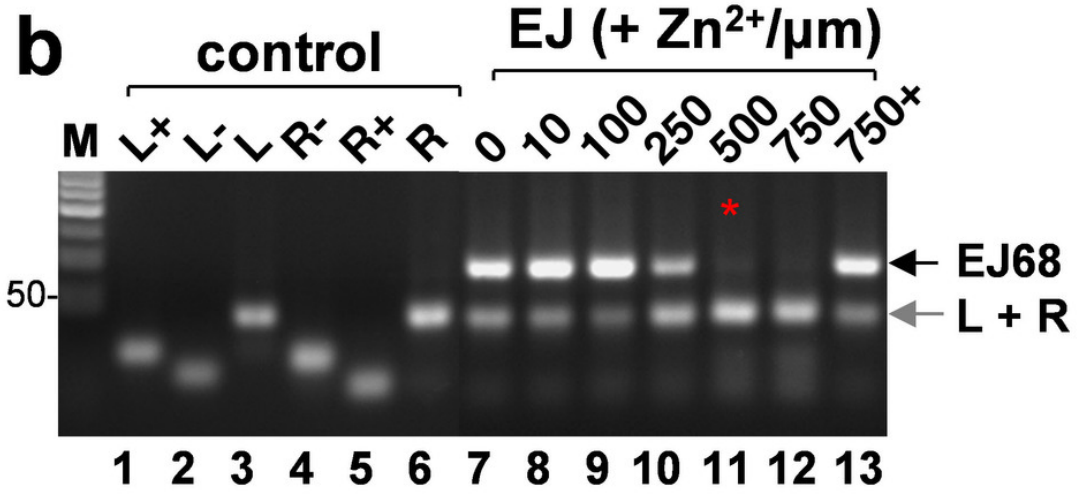
  --  sequenced delivery of user messages within multiple streams, with
      an option for order-of-arrival delivery of individual user
      messages,

  --  optional bundling of multiple user messages into a single SCTP
      packet, and
```

```
--  network-level fault tolerance through supporting of multi-homing
    at either or both ends of an association.

The design of SCTP includes appropriate congestion avoidance behavior
and resistance to flooding and masquerade attacks.
```





Element 9:

and a second module designed to start the timer assigned to the each of the fragments,

that we've correctly configured the loader, let's learn how to use it, starting by taking a look at the `require` function.

A Web framework is a collection of packages or modules which allow developers to write Web applications (see WebApplications) or services without having to handle such low-level details as protocols, sockets or process/thread management.

The majority of Web frameworks are exclusively server-side technology, although, with the increased prevalence of AJAX, some Web frameworks are beginning to include AJAX code that helps developers with the particularly tricky task of programming (client-side) the user's browser. At the extreme end of the client-side Web Frameworks is technology that can use the web browser as a full-blown application execution environment (a la gmail for example): see Web Browser Programming for details.

As a developer using a framework, you typically write code which conforms to some kind of conventions that lets you "plug in" to the framework, delegating responsibility for the communications, infrastructure and low-level stuff to the framework while concentrating on the logic of the application in your own code. This "plugging in" aspect of Web development is often seen as being in opposition to the classical distinction between programs and libraries, and the notion of a "mainloop" dispatching events to application code is very similar to that found in GUI programming.

Generally, frameworks provide support for a number of activities such as interpreting requests (getting form parameters, handling cookies and sessions), producing responses (presenting data as HTML or in other formats), storing data persistently, and so on. Since a non-trivial Web application will require a number of different kinds of abstractions, often stacked upon each other, those frameworks which attempt to provide a complete solution for applications are often known as **full-stack** frameworks in that they attempt to supply components for each layer in the stack.

Many frameworks now provide an element of customization in their support for the above activities and abstractions, utilizing components in that they provide abstractions only for certain specific things. As a result, it can be possible for you to build your own full-stack framework almost entirely from existing components.

Popular Full-Stack Frameworks

A web application may use a combination of a base HTTP application server, a storage mechanism such as a database, a template engine, a request dispatcher, an authentication module and an AJAX toolkit. These can be individual components or be provided together in a high-level framework.

These are the most popular high-level frameworks. Many of them include components listed on the WebComponents page.

Name	Latest version	Latest update date	description
Django	1.7.1	2014-10-22	The Web framework for perfectionists (with deadlines). Django makes it easier to build better Web apps more quickly and with less code. Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. It lets you build high-performing, elegant Web applications quickly. Django focuses on automating as much as possible and adhering to the DRY (Don't Repeat Yourself) principle. See Django
TurboGears	2.3.4	2014-10-03	the rapid Web development webframework you've been looking for. Combines SQLAlchemy (Model) or Ming (MongoDB Model), Genshi (View), Repoze and Tosca Widgets. Create a database-driven, ready-to-extend application in minutes. All with designer friendly templates, easy AJAX on the browser side and on the server side, with an incredibly powerful and flexible Object Relational Mapper (ORM), and with code that is as natural as writing a function. After reviewing the Documentation, check out the Tutorials
web2py	2.9.10	2014-09-15	* Python 2.4 to 2.7, Python 3.x friendly (compile but not tested no support yet) * All in one package with no further dependencies. Development, deployment, debugging, testing, database administration and maintenance of applications can be done via the provided web interface. * web2py has no configuration files, requires no installation, can be run off a USB drive. * web2py uses Python for the Model, View and the Controller * Built-in ticketing system to manage errors * Internationalization engine, caching system * Flexible authentication system (LDAP, MySQL, janrain etc) * NIX(Linux, BSD), Windows, Mac OSX, tested on EC2, Webfaction * works with

MySQL, PostgreSQL, SQLite , Firebird, Oracle, MSSQL and the Google App Engine via an ORM abstraction layer. * Includes libraries to handle HTML/XML, RSS, ATOM, CSV, RTF, JSON, AJAX, XMLRPC, WIKI markup. * Production ready, capable of upload/download of very large files * Emphasis on backward compatibility.

Element 10:

wherein the timer has a timeout value depending on a life of the each of the fragments

Chapter 5. Configuration

This covers the configuration for JBoss Remoting discovery, connectors, marshallers, and transports. All the configuration properties specified can be set either via calls to the object itself, including via JMX (so can be done via the JMX or Web console), or via a JBoss AS service xml file. Examples of service xml configurations can be seen with each of the sections below. There is also an example-service.xml file included in the remoting distribution that shows full examples of all the remoting configurations.

5.1. General transport configuration

Remoting offers a variety of ways of configuring transports on the server side and client side. This section presents an overview, and the rest of the chapter elaborates the material presented here. For easy reference the configuration parameters discussed throughout the chapter are gathered together at the end of the chapter in section Configuration by properties

5.1.1. Server side configuration

The heart of the server side is the Connector, and it is through the Connector that the server side of a transport is configured. The central goals of configuration on the server side are to establish a server invoker and supply it with a set of invocation handlers. Only one invoker can be declared per Connector. Although declaring an invocation handler is not required, it should only be omitted in the case of declaring a callback server that will not receive direct invocations, but only callback messages. Otherwise client invocations can not be processed. The invocation handler is the only interface that is required by the remoting framework for a user to implement and will be what the remoting framework calls upon when receiving invocations.

There are two general approaches to server side configuration: programmatic and declarative. A variety of programmatic techniques work in any environment, including the JBoss Application Server (JBossAS). Moreover, JBossAS adds the option of declarative configuration. In particular, the SARDeployer (see The JBoss 4 Application Server Guide on the labs.jboss.org web site) can read information from a *-service.xml file and use it to configure MBeans such as Connectors.

5.1.1.1. Programmatic configuration.

The simplest way to configure a Connector is to pass an InvokerLocator to a Connector constructor. For example, the code fragment

```
            String locatorURI = "socket://test.somedomain.com:8084";
            String params = "/?clientLeasePeriod=10000&timeout=120000";
            locatorURI += params;
            InvokerLocator locator = new InvokerLocator(locatorURI);
            Connector connector = new Connector(locator);
            connector.create();
            SampleInvocationHandler invocationHandler = new SampleInvocationHandler();
```

```
connector.addInvocationHandler("sample", invocationHandler);
connector.start();
```

creates a server invoker based on the socket transport, directs it to listen for invocations on port 8084 of host test.somedomain.com, and passes two configuration parameters, "clientLeasePeriod" and "timeout". It also supplies the server invoker with an invocation handler.

One limitation of the InvokerLocator is that it can only represent string values. An alternative that overcomes this limitation is to pass some or all of the parameters to the Connector by way of a configuration map. The following code fragment accomplishes all that the previous fragment does, but it passes one parameter by way of the InvokerLocator and passes the other by way of a configuration map. It also passes in a non-string object, a ServerSocketFactory:

```
String locatorURI = "socket://test.somedomain.com:8084";
String params = "/?clientLeasePeriod=10000";
locatorURI += params;
InvokerLocator locator = new InvokerLocator(locatorURI);
HashMap config = new HashMap();
config.put(ServerInvoker.TIMEOUT, 120000);
config.put(ServerInvoker.SERVER_SOCKET_FACTORY, new MyServerSocketFactory());
Connector connector = new Connector(locator, config);
connector.create();
SampleInvocationHandler invocationHandler = new SampleInvocationHandler();
connector.addInvocationHandler("sample", invocationHandler);
connector.start();
```

Note that the value of ServerInvoker.TIMEOUT is "timeout", and the value of ServerInvoker.SERVER_SOCKET_FACTORY is "serverSocketFactory". These configuration map keys are discussed throughout the chapter and accumulated in section Configuration by properties. Also, server socket factory configuration is covered in Socket factories and server socket factories.

A third programmatic option is available for those configuration properties which happen to be server invoker MBean properties. In the following fragment, the server invoker is obtained from the Connector and a ServerSocketFactory is passed to it by way of a setter method:

```
String locatorURI = "socket://test.somedomain.com:8084";
String params = "/?clientLeasePeriod=10000";
locatorURI += params;
InvokerLocator locator = new InvokerLocator(locatorURI);
HashMap config = new HashMap();
config.put(ServerInvoker.TIMEOUT, 120000);
Connector connector = new Connector(locator, config);
connector.create();
ServerInvoker serverInvoker = connector.getServerInvoker();
ServerSocketFactory ssf = new MyServerSocketFactory();
serverInvoker.setServerSocketFactory(ssf);
SampleInvocationHandler invocationHandler = new SampleInvocationHandler();
connector.addInvocationHandler("sample", invocationHandler);
connector.start();
```

Element 11:

and time parameters thereof

1. Field of the Invention

The present invention relates to an apparatus and a method for calculating gate delay used in logic simulation and timing analysis, and in particular, to an apparatus and a method for calculating gate delay using an RC model.

2. Description of the Background Art

Recently, the higher integration and multifunction of a semiconductor integrated circuit are increasing its circuit scale, thereby making the time required for its development longer. One method of reducing the time required for the development is to minimize re-designing and re-development due to the malfunction of a produced semiconductor integrated circuit. To achieve this goal, the simulation result obtained in a stage of designing a semiconductor integrated circuit must approximate to the timing of a signal between the gates within an actually produced semiconductor integrated circuit, and therefore there is an increasing need for a delay calculation apparatus with higher accuracy.

FIG. 1 is a block diagram showing a structure of a conventional delay calculation apparatus. The delay calculation apparatus includes: an input waveform gradient calculation apparatus **51** for extracting the gradient of the input waveform applied to an input terminal of a gate; an output load model calculation apparatus **52** for RC modeling the load applied to an output terminal of the gate; a gate delay calculation apparatus **53** for calculating the delay experienced from the input terminal to the output terminal of the gate; and a wire delay calculation apparatus**54** for calculating the delay (due to wiring) between the output terminal of the gate and the input terminal of a gate in a next stage. Gate delay data **55**calculated by gate delay calculation apparatus **53** and wire delay data **56**calculated by wire delay calculation apparatus **54** are used in logic simulation or timing analysis by a simulator or the like.

Element 12:

, wherein the timer is running until processing of the each of the fragments is terminated,

Abstract: This invention relates to techniques for managing the transmission and reception of data fragments that contains one or more data blocks using a single timer. One embodiment of the invention includes the following steps: processing the fragments sequentially, wherein each fragment has a processing index that corresponds to sequential processing of that fragment; processing each of the fragments until a termination upon meeting a first pre-defined condition; assigning a timer to an un-terminated fragment having a lowest processing index; starting said timer having a timeout value; and running said timer until the processing of said un-terminated fragment is terminated. This invention uses only one timer for each connection and therefore reduces memory and operational needs in the management of the data fragments that are being received or transmitted.
Type: Grant
Filed: August 26, 2008
Issued: February 26, 2013
Assignee: Sun Management, LLC
Inventors: Yalun Li, William Li

Element 13:

the timer is then reassigned to a next one of the fragments before the next one of the fragments is processed.

Problems that crop up intermittently in writing, often referred to as "faulty mechanics" by English teachers, might be thought of as sporadic or intermittent noise. It tends to occur occasionally, rather than affect every page the way a poor choice of type size or confused organization of material might. Of course enough sporadic noise in a document, such as repeated misspellings or numerous sentence fragments, can easily turn into constant noise and give your reader an impression of hastily and carelessly produced work undeserving of the response or feedback you want.

To help you eliminate sporadic noise this guide looks at where it is most likely to occur in spelling, punctuation, sentence structure, and technical usage. We also give some pointers on how to edit your writing in order to remove this kind of noise.

SPELLING AND SPELL CHECKERS

You might think the spell checkers that come with most word processing software have eliminated any need to be a careful speller. Unfortunately this is not the case. With apologies to Shakespeare we took his words "A rose by any other name would smell as sweet," and ran them through a spell checker as *A nose by any outer dame wood small as sweat*. No red flags were raised by the program. Nor will spell checkers catch common errors such as confusing *there* for *their*, or *to* for *too*. Some typographical errors simple give you other words which will pass unnoticed, as in this sentence. A very slight slip of the finger on the keyboard can make the difference between asking for some forms to be mailed to you or nailed to you. A quick transposition could render a memorandum *nuclear* rather than simply *unclear*.

At best, the effect of poor spelling on your readers is a sense of annoyance, or at least of having their attention distracted by something other than what you want to communicate. At worst, noise created by spelling glitches can bring readers to a stop and cause them to seriously question your ability as a writer. They might even suspect that a person who is careless with spelling could also be inept in more critical technical matters.

To reduce or eliminate any noise in your writing caused by incorrect spelling, use a spell checker but also have a standard dictionary nearby. A current dictionary is the only resource that can reliably answer questions such as

- whether there is more than one way to spell a word, or what the accepted plural forms of words such as *appendix* or *matrix* are
- how words like *well-known* or *so-called* are hyphenated, or whether a computer is *on-line* or *online*
- whether it is appropriate to write about *FORTRAN, Fortran,* or *fortran*
- what the difference between British and American spelling or usage might be
- what the accepted past tense is of recent verbs that have come into technical English such as *input*

It is especially important for an engineer to use a current dictionary. English is a dynamic language, and the language of science and technology changes even more rapidly as knowledge increases and devices are developed. You won't find words like *software*, *modem*, and *LED* in a dictionary from the 1950s, and since then older words such as *bug*, *hardware*, *interface*, and even *mouse* have taken on new meanings. Some usage has yet to be decided on: Would a computer shop advertise that it repairs *mice* or *mouses*? Do you send *e-mail, E-mail*, or *email*? (Right now all three options are used.)

PUNCTUATION

Would you want to drive on a busy highway or in a city where there were no traffic signs? Controlling the flow of traffic is vital if anyone is to get anywhere. Similarly, within sentences the flow of meaning is controlled by punctuation marks, the conventionally agreed-upon "traffic signals" of written communication. We do the same thing in spoken language by means of pitch, breath pauses, and emphasis. Directing the flow of ideas in writing is not really difficult, and a useful procedure when you're unsure of how to punctuate a sentence might be to say it aloud as in normal conversation. Pay careful attention to where you pause naturally within the sentence-that is likely to be where you need some punctuation.

Many detailed guides to punctuation exist, and you may want to look at them if you have a lot of queries in this area. You will also find excellent advice on punctuation in the front or back sections of some standard college dictionaries. Meanwhile, the following suggestions are offered on the most common problems many engineers tend to have with punctuation.

* * * * *

End of Report

Please rank my report. The following link leads to our profile at ClaimCharts.info, where you could rank our report.
Rank Here

Evaluation Code: MTZANUAyNkAyMg==

We are available to answer follow up questions, provide clarifications and search for additional materials upon request. For public contact - please refer to our profile at ClaimCharts.info. For private contact - please call/email us.

Our hourly rate is: $

Non-Disclosure Agreement:

We understand that for the purpose of assisting you in collecting data, you may disclose to us certain confidential information that you want to keep confidential; therefore, for good and valuable consideration, the receipt and sufficiency of which is hereby acknowledged, we agree as follows:

1. For a period of twelve (36) months from the date we receive your confidential information ("Effective Date"), we will not use or disclose the information, except: (a) in the event that such information (i) comes into the public domain not through us; (ii) is already known to us at the time of disclosure; (iii) is lawfully otherwise learnt or independently developed by us; or (iv) is required to be disclosed by law.

2. Since a breach of our obligations hereunder may result in irreparable harm to you, we agree that upon such breach or threat thereof, you shall be entitled to specific performance and other injunctive relief without posting a bond, and in the event of disclosures required by court order, we shall promptly notify you of the proceeding.

3. This agreement supersedes all prior discussions and writings and constitutes the entire agreement between you and us with respect to the subject matter hereof. This Agreement shall be binding upon the successors and assigns of the respective parties, and may not be modified except by an instrument executed by both parties.

Alon Anthony